

---

## Trabajo Final Integrador

---

Materia:

Virtualización: Consolidación de Servidores

Docente:

Ing. Luis María Carriles

Alumna:

Albornoz, Silvia Melisa

DNI: 45 275 151

Legajo: 56113

Tucumán, 2026

## Índice

---

|                                                                                    |          |
|------------------------------------------------------------------------------------|----------|
| <b>1. Introducción</b>                                                             | <b>2</b> |
| <b>2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE</b>           | <b>2</b> |
| <b>3. Fase 1: Preparación del Código en Entorno Local</b>                          | <b>2</b> |
| 3.1. Estructura del Proyecto . . . . .                                             | 3        |
| 3.2. Dependencias de Producción . . . . .                                          | 3        |
| <b>4. Fase 2: Transferencia de Archivos al Contenedor</b>                          | <b>3</b> |
| <b>5. Fase 3: Despliegue de la Aplicación Flask (LXC 167)</b>                      | <b>4</b> |
| 5.1. Aislamiento del Entorno Python . . . . .                                      | 4        |
| 5.2. Lanzamiento del Servidor en Segundo Plano . . . . .                           | 4        |
| <b>6. Fase 4: Configuración de PostgreSQL (LXC 168)</b>                            | <b>4</b> |
| 6.1. Inicialización del Motor de Base de Datos . . . . .                           | 4        |
| 6.2. Aprovisionamiento de Entidades Relacionales y Credenciales . . . . .          | 5        |
| 6.3. Concesión de Permisos de Conexión Remota e Inter-LXC . . . . .                | 5        |
| <b>7. Fase 5: Configuración del Proxy Inverso y Estabilidad del Servicio (NAP)</b> | <b>6</b> |
| 7.1. Estabilidad y Persistencia en el Entorno Institucional . . . . .              | 6        |
| 7.2. Integración de Cabeceras HTTP mediante ProxyFix . . . . .                     | 6        |
| 7.3. Lógica de Descarga Forzada del Informe de Fin de Curso . . . . .              | 6        |
| <b>8. Conclusión</b>                                                               | <b>7</b> |

## 1. Introducción

El presente documento expone los criterios de diseño, las contingencias técnicas encontradas y el procedimiento paso a paso ejecutado para el despliegue de un **Blog Personal** con arquitectura cliente-servidor desacoplada, compuesta por una aplicación web desarrollada en **Flask/Python** y un sistema de persistencia relacional basado en **PostgreSQL**.

El entorno de destino corresponde a la infraestructura de la **NAP** (*Núcleo de Asistencia de Redes*) de la facultad, accesible a través del dominio `nap.frt.utn.edu.ar`. El despliegue representó un desafío crítico de administración de sistemas debido a dos restricciones principales:

- **Limitación de memoria RAM:** límite estricto de 128 MB por contenedor LXC.
- **Aislamiento de red:** la subred privada universitaria (172.16.90.X) impide el acceso directo entrante desde el exterior.

## 2. Creación y Aprovisionamiento de Contenedores LXC en Proxmox VE

Para garantizar el aislamiento de procesos y optimizar el uso de los recursos del hipervisor, se aprovisionaron **dos contenedores Linux (LXC) independientes** sobre la distribución **Debian 12 Bookworm**, cada uno con responsabilidad única (principio de *single responsibility*):

| Parámetro        | LXC Flask (Aplicación) | LXC PostgreSQL (BD)  |
|------------------|------------------------|----------------------|
| Hostname         | 45275151A              | 45275151DB           |
| Plantilla de OS  | debian-12-standard     | debian-12-standard   |
| vCPU             | 1 núcleo               | 1 núcleo             |
| Memoria RAM      | 128 MB                 | 128 MB               |
| Memoria Swap     | 0 MB                   | 0 MB                 |
| Almacenamiento   | 8 GB (ext4)            | 8 GB (ext4)          |
| Interfaz de red  | eth0 (Bridge: vmbr0)   | eth0 (Bridge: vmbr0) |
| Dirección IP     | 172.16.90.167/24       | 172.16.90.168/24     |
| Puerta de enlace | 172.16.90.1            | 172.16.90.1          |

Tabla 1: Especificaciones de los contenedores LXC aprovisionados en Proxmox VE.

## 3. Fase 1: Preparación del Código en Entorno Local

La aplicación web fue desarrollada en Python utilizando el *framework* **Flask** con el driver nativo **psycopg2** para la interacción con la base de datos PostgreSQL.

### 3.1. Estructura del Proyecto

El repositorio local se organizó respetando la jerarquía estándar de Flask para activos estáticos y plantillas Jinja2:

```
1 blog/
2     app.py
3     static/
4         uploads/
5             Informe_TPF.pdf
6     templates/
7         index.html
8         admin.html
```

Imagen 1: Estructura del repositorio de la aplicación.

### 3.2. Dependencias de Producción

Las librerías requeridas se fijaron con versiones específicas en el archivo `requirements.txt` para garantizar la reproducibilidad del entorno:

```
1 Flask==2.3.2
2 psycpg2-binary==2.9.6
3 Werkzeug==2.3.6
```

Imagen 2: Contenido de `requirements.txt`.

## 4. Fase 2: Transferencia de Archivos al Contenedor

Dado que el aislamiento perimetral de la red universitaria imposibilitaba el acceso directo entrante vía SSH/SFTP desde una ubicación externa, se utilizó **GitHub** como pasarela de transferencia intermedia bajo la cuenta de usuario AlbMelisa.

**Procedimiento en el equipo local:**

```
1 cd ~/blog
2 git init
3 git add .
4 git commit -m "Initial commit: blog personal Flask"
5 git remote add origin https://github.com/AlbMelisa/tfi-blog-flask.git
6 git push -u origin main
```

Imagen 3: Inicialización del repositorio y carga a GitHub.

**Procedimiento en el contenedor LXC 167** (con salida nativa a Internet a través del puerto 443/HTTPS):

```
1 cd /opt
2 git clone https://github.com/AlbMelisa/tfi-blog-flask.git
```

Imagen 4: Clonado del repositorio dentro del contenedor de aplicación.

## 5. Fase 3: Despliegue de la Aplicación Flask (LXC 167)

---

### 5.1. Aislamiento del Entorno Python

Para evitar conflictos con los paquetes del sistema operativo Debian 12, se instaló la utilidad de entornos virtuales y se inicializó un espacio aislado de ejecución:

```
1 apt install python3-venv -y
2 cd /opt/blog
3 python3 -m venv venv
4 source venv/bin/activate
```

Imagen 5: Creación y activación del entorno virtual Python.

Con el entorno activo (indicado por el prefijo (**venv**)), se actualizaron las herramientas de empaquetado y se instalaron las dependencias del *backend*:

```
1 pip install --upgrade pip
2 pip install -r requirements.txt
```

Imagen 6: Instalación de dependencias en el entorno virtual.

### 5.2. Lanzamiento del Servidor en Segundo Plano

Para asegurar la persistencia del proceso web de Flask y evitar su finalización al cerrar la terminal interactiva del hipervisor Proxmox, se ejecutó la aplicación utilizando **nohup**, que desvincula el proceso del terminal de control:

```
1 nohup python3 app.py > /opt/blog/flask.log 2>&1 &
2 echo "PID: $!"
```

Imagen 7: Ejecución persistente del servidor Flask.

**Nota técnica:** La redirección `>/opt/blog/flask.log 2>&1` consolida la salida estándar y los errores en un único archivo de log, facilitando el diagnóstico posterior sin necesidad de mantener una sesión interactiva abierta.

## 6. Fase 4: Configuración de PostgreSQL (LXC 168)

---

### 6.1. Inicialización del Motor de Base de Datos

Desde la consola del contenedor dedicado a persistencia (45275151DB), se instalaron los paquetes binarios del motor relacional:

```
1 apt update && apt install postgresql postgresql-contrib -y
2 systemctl enable postgresql
3 systemctl start postgresql
```

Imagen 8: Instalación de PostgreSQL en el contenedor de base de datos.

## 6.2. Aprovisionamiento de Entidades Relacionales y Credenciales

Se accedió a la consola interactiva `psql` adoptando los privilegios del superusuario del sistema:

```
1 su - postgres
2 psql
```

Imagen 9: Acceso al intérprete de comandos `psql`.

Dentro del prompt `postgres=#`, se configuraron el usuario de aplicación, la base de datos dedicada, la codificación de caracteres y la zona horaria regional:

```
1 CREATE USER postgres WITH PASSWORD '45275151';
2
3 CREATE DATABASE blog_db
4     WITH OWNER          = postgres
5         ENCODING        = 'UTF8'
6         LC_COLLATE      = 'es_AR.UTF-8'
7         LC_CTYPE        = 'es_AR.UTF-8';
8
9 GRANT ALL PRIVILEGES ON DATABASE blog_db TO postgres;
```

Imagen 10: Creación de usuario, base de datos y configuración regional.

## 6.3. Concesión de Permisos de Conexión Remota e Inter-LXC

Por defecto, PostgreSQL restringe la escucha de *sockets* únicamente a la interfaz local (`localhost`). Para habilitar la comunicación desde el servidor de aplicación fue necesario modificar dos archivos de configuración.

**1. Edición de `postgresql.conf`** — forzar la escucha en todas las interfaces de red disponibles:

```
1 # /etc/postgresql/<version>/main/postgresql.conf
2 listen_addresses = '*'
```

Imagen 11: Modificación del parámetro `listen_addresses`.

**2. Edición de `pg_hba.conf`** — autorizar conexiones provenientes exclusivamente de la subred del segmento NAP:

| # | TYPE | DATABASE | USER     | ADDRESS        | METHOD |
|---|------|----------|----------|----------------|--------|
| 2 | host | blog_db  | postgres | 172.16.90.0/24 | md5    |

Imagen 12: Regla de autenticación en `pg_hba.conf`.

**3. Aplicación de los cambios** mediante reinicio estructural del servicio:

```
1 systemctl restart postgresql
2 systemctl status postgresql
```

Imagen 13: Reinicio del servicio PostgreSQL para aplicar la nueva configuración.

## 7. Fase 5: Configuración del Proxy Inverso y Estabilidad del Servicio (NAP)

---

### 7.1. Estabilidad y Persistencia en el Entorno Institucional

Para mantener levantado de forma robusta el blog bajo la infraestructura perimetral de la NAP, el tráfico externo es administrado por un servidor de la facultad que actúa como proxy inverso. Con el objetivo de garantizar la estabilidad de la plataforma, el entorno redirige las peticiones externas resueltas bajo el dominio `nap.frt.utn.edu.ar` acoplándolas a la subruta dedicada basada en el documento de identidad del alumno (`/45275151/`).

### 7.2. Integración de Cabeceras HTTP mediante ProxyFix

Debido a que el servidor perimetral de la NAP remapea las peticiones web externas hacia el puerto interno 5000 del contenedor 45275151A, se identificó una pérdida de contexto original en las cabeceras de transmisión relativas al protocolo seguro HTTPS y a la ruta base de subdirectorio. Para resolver esta anomalía y asegurar que el ruteo interno opere en perfecta sincronía, se integró el middleware nativo ProxyFix provisto por la librería Werkzeug en el script principal `app.py`:

```
1 from werkzeug.middleware.proxy_fix import ProxyFix
2
3 app = Flask(__name__)
4 app.wsgi_app = ProxyFix(app.wsgi_app, x_for=1, x_proto=1,
    x_host=1, x_prefix=1)
```

Imagen 14: Habilitación de ProxyFix para la asimilación de cabeceras inversas.

Esta configuración obliga a Flask a procesar variables críticas como `X-Forwarded-Proto` y `X-Forwarded-Prefix`, permitiendo que la aplicación reconozca solicitudes HTTPS y resuelva las URLs de forma absoluta dentro de los límites del subdirectorio institucional asignado.

### 7.3. Lógica de Descarga Forzada del Informe de Fin de Curso

Para el botón de descarga del documento técnico integrado en la interfaz del blog, se implementó un endpoint exclusivo en el backend que encapsula la descarga directa como adjunto binario, previniendo que los navegadores fueren la apertura en línea del archivo:

```
1 from flask import send_from_directory
2
3 @app.route("/download-pdf")
4 def download_pdf():
5     return send_from_directory(
6         app.config["UPLOAD_FOLDER"],
7         "Informe_TPF.pdf",
8         as_attachment=True,
```

```
9         download_name="TFI_Virtualizacion_AlbornozSilviaMelisa  
10         .pdf "  
        )
```

Imagen 15: Ruta dedicada a la descarga directa del archivo adjunto.

## 8. Conclusión

---

La arquitectura propuesta fue implementada de forma exitosa, cumpliendo con la totalidad de los requisitos funcionales y técnicos definidos para el proyecto. Los principales logros obtenidos fueron:

- **Separación de responsabilidades:** la distribución de los servicios en nodos LXC independientes permitió una gestión más eficiente de los recursos y facilitó el diagnóstico de fallos.
- **Operación dentro de restricciones de hardware:** la aplicación y la base de datos operan de forma estable con tan solo 128 MB de RAM cada una, demostrando la eficiencia del stack seleccionado.
- **Conectividad segura inter-contenedor:** la configuración de PostgreSQL restringe el acceso remoto únicamente a la subred interna 172.16.90.0/24, reduciendo la superficie de ataque.
- **Persistencia del servicio web:** el uso de `nohup` garantiza la disponibilidad del servidor Flask de forma independiente al ciclo de vida de las sesiones interactivas.

En síntesis, la infraestructura desplegada constituye una solución confiable, organizada y escalable, capaz de satisfacer los requerimientos actuales del proyecto y de servir como base para futuras ampliaciones, tales como la configuración de un servicio *systemd* para la gestión del ciclo de vida del proceso Flask.